ROS2 Humble

Adithi Pillai
Department of Computer Science and Engineering
Manipal Institute of Technology
Manipal, India
adithi.mitmpl2026@learner.manipal.edu

***Abstract*— This report documents the exploration of the fundamentals of ROS 2 Humble. The official Humble documentation was used to study concepts like the computation graph, nodes, topics, services, actions, parameters, custom interfaces, colcon workspaces and QoS settings. It also includes solutions and graphs for the two questions given.**

Keywords—ROS2, computation graph, topics, services, actions, parameters, custom interfaces, colcon, Quality of Service.

INTRODUCTION

ROS2 is a set of tools and libraries that help developers create robots using various platforms and programming languages. ROS2 consists of basic components like nodes, interfaces (topics, services and actions) and parameters. This report also covers topics like custom workspaces, colcon, workspace structure and QoS settings. It also includes solutions and graphs for the two questions given.

The aim is to give a clear, practical understanding of how a ROS 2 system is structured, built and inspected, so that the same patterns can be reused in larger robotic applications.

THE ROS 2 COMPUTATION GRAPH

The ROS 2 computation graph is the network of elements that exchange and process data in a running system. It includes every node and the connections between them such as topics, services, actions and parameters. It can be visualised using the command rqt graph.

Nodes

Each node in ROS should be responsible for a single, modular purpose. A node can communicate to other nodes through topics, services, actions, or parameters. A node is created using: ros2 run <package_name> <executable_name>. ros2 node list will show you the names of all running nodes.

Topics

A topic acts as a communication channel between two nodes to exchange messages. A node can publish data to any number of topics and simultaneously have subscriptions to any number of topics. Every topic has a name and a message type, and publishers and subscribers do not know about each other. The message type is how nodes know they are talking about the same information.

Services

Services are based on a call-and-response model versus the publisher-subscriber model of topics. While topics send data continuously, services only provide data when it is requested by a client. There can be many service clients using a service but only one service server. The service type has a request part and a response part, separated by --- in its .srv file.

Actions

Actions are used for long-running tasks. They consist of three parts: a goal, feedback, and a result. They are like a combination of topics and services. The client sends a goal and requests the result through services, and the server publishes feedback on topics. Unlike a service, an action can be cancelled while executing and reports progress. Actions also have types like topics and services. The action type has a request, result and feedback part where --- separates each section.

Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, booleans, strings, and lists. In ROS 2, each node maintains its own parameters.

TOPIC VERSUS SERVICE

TABLE I. COMPARISON OF TOPICS AND SERVICES

Aspect	**Topic**	**Service**
Pattern	Publisher/subscriber, one-way	Request/response, two-way
Cardinality	Many publishers, many subscribers	Many clients, one server
Data	Continuous stream, no reply	On demand; caller waits for one reply
Connection	Anonymous, publisher and subscriber don't know each other	Client must know the service name
File	.msg	.srv

Topics should be used while dealing with continuous streams of data like in camera feeds. Services should be used for request/response communication. They are ideal for one-off commands like checking battery. Actions are used for long running processes that have a clear beginning and end.

CUSTOM INTERFACES

ROS 2 has some standard interfaces such as std_msgs and geometry_msgs. However, sometimes applications need their own. Interfaces are described in plain text files and are described as .msg for messages, .srv for services and .action for actions. , placed in msg, srv or action folders inside a package. You can create your own custom interfaces in an ament_cmake package, and then use it in a C++ or Python node.

QUALITY OF SERVICE SETTINGS

QoS profiles in ROS 2 are basically rules that decide how messages should be delivered through a topic. A publisher offers a QoS profile and a subscriber requests one. They are connected only if what's offered meets what's requested. If it doesn't, they don't connect, and no messages flow between them. Some examples of elements of these profiles are history which decides whether to keep last few messages or all messages and depth which decides how many messages to keep if last few messages are kept.

COLCON AND WORKSPACE STRUCTURE

A workspace is a directory containing ROS2 packages. colcon is the build tool used by ROS2. It takes the ROS packages and builds them so ROS2 can use them. A workspace layered on top of the installed ROS2 distribution is called an overlay, and the distribution is the underlay.

The workspace contains an src folder which is the only one used by the developer. colcon also creates build, install and log folders. Inside src, each package has its own folder containing a package.xml file. C++ (ament_cmake) packages also contain a CMakeLists.txt, while Python (ament_python) packages contain a setup.py and setup.cfg instead. Packages that define custom interfaces additionally have msg, srv or action folders holding the interface definition files.

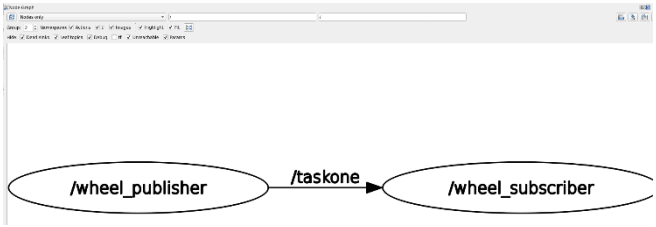
SOLUTIONS WITH ROS GRAPHS

First Question

A package named taskone was created in ros2_ws/src using the ament_cmake build type (ros2 pkg create --build-type ament_cmake --license Apache-2.0 taskone).

A msg directory was added containing TaskOne.msg, which holds the two float values angvel and radius. CMakeLists.txt and package.xml were edited to add the message generation dependencies (rosidl_default_generators, rosidl_default_runtime) and the rosidl_interface_packages group, so that TaskOne.msg is converted into usable code during the build. The package was then built with colcon build --packages-select taskone and sourced.

A Python package, taskone_nodes, was created with rclpy and taskone as dependencies (ros2 pkg create --build-type ament_python taskone_nodes --dependencies rclpy taskone). The files publisher.py and subscriber.py were written in the inner taskone_nodes directory, and both import the message with from taskone.msg import TaskOne. The publisher reads the angular velocity and radius from the terminal, stores them in a TaskOne message, and publishes it to /taskone. The subscriber subscribes to the topic /taskone, calculates the longitudinal speed as $v = \omega \times r$ and prints it in the terminal.

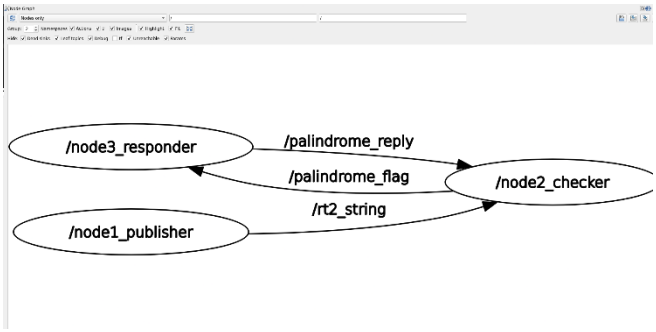


Second Question

A package named rt2 was created in ros2_ws/src using the ament_cmake build type (ros2 pkg create --build-type ament_cmake rt2). A msg directory was added containing RT2String.msg, which holds a single string value named data. CMakeLists.txt and package.xml were edited to add the message generation dependencies (rosidl_default_generators, rosidl_default_runtime) and the rosidl_interface_packages group, so that RT2String.msg is converted into usable code during the build.

A Python package, rt2_nodes, was then created with rclpy, std_msgs and rt2 as dependencies (ros2 pkg create --build-type ament_python rt2_nodes --dependencies rclpy std_msgs rt2). The files node1.py, node2.py and node3.py were written in the inner rt2_nodes directory and registered as executables in setup.py, and the packages were built with colcon build --packages-select rt2 rt2_nodes and sourced.

Node 1 reads a string from the terminal, stores it in an RT2String message, and publishes it to /rt2_string. Node 2 subscribes to /rt2_string, checks whether the string is a palindrome by comparing it with its reverse, and publishes 1 (true) or 0 (false) as an Int32 message to /palindrome_flag. Node 3 subscribes to /palindrome_flag and publishes “Yes” if the value is 1 or “No” if it is 0 as a String message to /palindrome_reply. Node 2 subscribes to /palindrome_reply and prints the received answer in the terminal.



CONCLUSION

This report explored the fundamental concepts of ROS2, from the computation graph and nodes to topics, services, actions and parameters. The differences between topics and services, along with custom interfaces, colcon, workspace structure and QoS settings, were also examined, showing how ROS 2 gives developers flexible control over how nodes communicate. The two solutions demonstrated how these concepts work together to build a connected system.

ACKNOWLEDGEMENT

I would like to thank the Driverless subsystem seniors for providing their guidance and resources that helped in completing the task and understanding the topic.

REFERENCES

https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html

https://www.youtube.com/watch?v=F3n0SMAFheM